Continuous Medical Filtering Benchmark: An Extension of Filtering Techniques for Entity Resolution in Medical Data

Adam Ben Rejeb, Kapil Kumar Khatri, and Vincent Melvin

Gottfried Wilhelm Leibniz Universität Hannover, Welfengarten 1, 30167 Hannover
{adam.ben.rejeb, kapil.kumar.khatri, melvin.vincent}@stud.uni-hannover.de

Abstract. Entity Resolution (ER) in medical data is complicated by domain-specific terminology, heterogeneous record structures, and large data volumes, yet existing filtering benchmarks focus almost exclusively on general-purpose datasets. We extend the Continuous Filtering Benchmark to the medical domain, evaluating three families of techniques: parameter-free and fine-tuned blocking workflows, sparse nearest-neighbor methods (ε-Join, kNN-Join), and dense ones (FAISS, ScaNN, DeepBlocker), across nine datasets spanning synthetic patient records, electronic health records, biomedical text mentions, physician payment records, and medical ontologies, with ground-truth sizes from 500 to 643,166 pairs.

No single approach dominates. Fine-tuned blocking workflows achieve the best F_1 on structured datasets (1.000 on FEBRL-4, 0.982 on Synthea, 0.997 on MedMentions), while kNN-Join also reaches 1.000 on FEBRL-1 and FEBRL-4. General-purpose dense embeddings (FAISS, ScaNN) underperform syntactic methods on seven of nine datasets, suggesting that medical ER needs domain-specific or ontology-aware representations. UMLS is the hardest case: its scale, duplicate density, and short synonym-dense concept strings hold every method to F_1=0.53 at best, and that on a 50% subset. Code and reproduction steps are available at https://github.com/KapKr21/ContinuousMedicalFilteringBenchmark

Keywords: Entity Resolution · Filtering Techniques · Blocking · Medical Data · Nearest Neighbor Methods · Benchmark · Meta-blocking

1 Introduction

Entity Resolution (ER), the task of identifying records that refer to the same real-world entity across one or more data sources, is a fundamental problem in data integration. In the medical domain, ER is particularly critical: duplicate patient records across hospital systems can lead to medication errors, redundant procedures, and fragmented care histories. Similarly, linking biomedical concepts across ontologies (e.g., UMLS, RxNorm) enables downstream applications such as clinical decision support, pharmacovigilance, and cohort identification for clinical trials.

Naively comparing all record pairs is quadratic in the number of entities and computationally prohibitive at real-world scale. *Filtering techniques* (also called blocking or candidate generation) address this by reducing the candidate pair space to a manageable subset before detailed matching is performed (Fig. 1). Because pairs missed during filtering cannot be recovered later, filtering effectiveness bounds the recall of the entire ER pipeline.

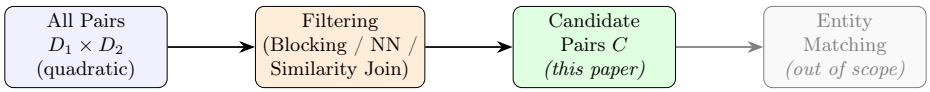

Fig. 1. The Entity Resolution filtering pipeline. Filtering reduces all pairs to a much smaller candidate set C, which the matching step then resolves into true duplicates.

Papadakis et al. [1] and, in extended journal form, Neuhof et al. [2] introduced the Continuous Filtering Benchmark, the first systematic comparison of *blocking* and *nearest-neighbor (NN) workflows*, the latter further split into *sparse* syntactic methods (e.g., kNN-Join, ε-Join) and *dense* learned-embedding methods (e.g., FAISS, ScaNN, DeepBlocker), across ten general-purpose datasets. We adopt this same taxonomy (Fig. 2).

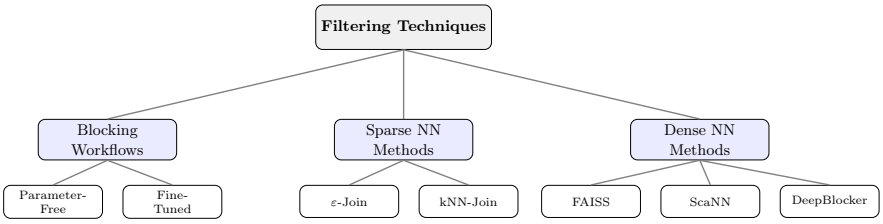

Fig. 2. Taxonomy of the three filtering families evaluated here, following the blocking-vs.-nearest-neighbor split of Papadakis et al. [1] and Neuhof et al. [2], with nearest-neighbor methods split further into sparse (syntactic) and dense (semantic) representations.

This benchmark's findings, however, do not directly transfer to the medical domain, whose records exhibit properties that challenge standard filtering assumptions:

- **Domain-specific vocabulary:** drug names, diagnosis codes, and anatomical terms are underrepresented in general-purpose language models, weakening pre-trained embeddings.
- **Short, homogeneous strings:** ontology datasets like UMLS contain short concept strings (e.g., "Aspirin" vs. "Acetylsalicylic acid") where non-matches overlap heavily and true matches may not, due to synonymy.
- **Scale and duplicate density:** datasets range from ∼800-record drug terminologies to ontology mappings of ∼135K×157K records with 643K ground-truth pairs.

Main Contributions

1. **A medical filtering benchmark:** to our knowledge, the first comprehensive evaluation of blocking, sparse NN, and dense NN filtering on nine medical ER datasets, establishing baselines for future research.
2. **Parameter sensitivity:** fine-tuning blocking workflows yields gains of up to 70× on Synthea and 6.6× on CMS over parameter-free defaults, showing that one-size-fits-all configurations are inadequate for medical data.
3. **Sparse vs. dense representations:** syntactic methods outperform general-purpose dense embeddings on seven of nine datasets, pointing to a domain-vocabulary gap in pre-trained sentence transformers.
4. **A hard case:** UMLS resists every method, with no full-collection result above F_1=0.31 and a reduced 50% subset topping out at 0.53, owing to its scale, duplicate density, and short ontology terms.
5. **Reproducibility findings:** implementation-level failure modes invisible in aggregate numbers, notably a tensor-shape mismatch in DeepBlocker's Hybrid model and a collapse of both approximate indices on CMS that exact search avoids.

2 Related Work

2.1 Entity Resolution and Filtering Techniques

Entity Resolution (ER) has long been studied as record linkage, deduplication, or reference reconciliation [**?**]. Because pairwise comparison scales quadratically, filtering (blocking) discards most non-matching pairs before any expensive comparison [3]. Classic methods build signatures from attribute values (tokens, character n-grams, sorted-neighborhood windows) and block entities sharing one. Meta-blocking [4] then restructures these blocks into a weighted candidate-pair graph and prunes low-weight edges, via Weighted Edge Pruning, (Reciprocal) Cardinality Node Pruning, or the schema-aware BLAST algorithm [5]. With block purging and filtering, these form the *blocking workflow* paradigm implemented in JedAI [6], used throughout this work.

A separate line formulates filtering as a *similarity join*: enumerate every pair whose similarity exceeds a threshold ε [9]. Inverted-index joins (ε-Join) and their top-k variant (kNN-Join) trade a tunable threshold for completeness guarantees under the chosen similarity function and representation.

2.2 Nearest-Neighbor and Embedding-Based Methods for Entity Resolution

Dense NN filtering embeds each entity as a fixed-length vector and retrieves candidates via approximate nearest-neighbor search. FAISS [10] offers exact (flat) and approximate graph-based (HNSW) search, trading a little recall for much

faster queries. ScaNN [11] uses a learned tree partition and anisotropic quantization, exposing the same brute-force-vs.-asymmetric-hashing trade-off. Both are agnostic to how vectors are produced; following the base benchmark, we use general-purpose sentence embeddings (all-MiniLM-L6-v2 [13]) rather than domain-specific ones. DeepBlocker [12] instead learns the tuple embedding itself, via an unsupervised AutoEncoder, a self-supervised Cross-Tuple Training (CTT) objective, or a Hybrid of both, before exact top-k pairing.

2.3 Benchmarking Filtering Techniques for Entity Resolution

Blocking workflows and NN methods were rarely compared under a common protocol until Papadakis et al. [1] and, in extended form, Neuhof et al. [2]. Their Continuous Filtering Benchmark evaluates both families across ten established ER datasets under schema-agnostic and schema-aware settings, reporting best precision at fixed recall targets $\tau \in \{0.85, 0.90, 0.95\}$ rather than a single operating point, since each technique's configuration space (filtering ratios, weighting schemes, thresholds, k) has a first-order effect on the recall-precision trade-off. They find that no paradigm dominates: blocking is most robust overall, sparse NN (kNN-Join) overtakes it on long, clean textual attributes, and dense NN is consistently weakest, penalized by the domain gap between general-purpose training corpora and dataset vocabulary.

We follow the same philosophy (parameter-free baselines, fine-tuned configurations, a common harness) but apply it to the medical domain, absent from the original ten bibliographic/e-commerce datasets, which we also use as a reproduction study (Section 5) to validate our pipeline.

2.4 Entity Resolution in the Medical and Biomedical Domain

Medical ER spans patient-record deduplication, linking de-identified health records, and aligning ontology concepts across vocabularies such as UMLS [18] and RxNorm [20]. Existing surveys [3] address filtering here only in passing.

On the patient-record side, filtering underpins privacy-preserving record linkage: Randall et al. [7] scale inverted-index and sorted-neighborhood blocking to hospital databases of millions of records, under encoding constraints our non-private methods avoid. Febrl [14] was built for, and remains widely used in, health and census linkage, akin to our FEBRL and Synthea [19] datasets.

On the ontology side, filtering overlaps with biomedical entity linking: mapping a mention to its canonical UMLS concept, as in MedMentions [15], is structurally Clean-Clean ER. SapBERT [8] learns dense embeddings aligned to UMLS synonym clusters, unlike the general-purpose embeddings we evaluate; whether this closes the sparse-over-dense gap we observe is left to future work. To our knowledge, ours is the first systematic, multi-method quantification of how these medical-domain properties affect filtering effectiveness.

3 Medical Datasets

We assemble nine medical and biomedical ER datasets, summarized in Table 1, spanning three orders of magnitude in scale (500 to over 600K ground-truth pairs) and both major ER task formulations: *Dirty ER*, where duplicates must be identified within a single, possibly noisy, collection ($D = D_1 = D_2$), and *Clean-Clean ER*, where two internally clean collections must be linked against each other.

Table 1. Overview of the nine medical datasets used in this benchmark.

Dataset	Domain	Source	$\lvert D_1 \rvert$	$\lvert D_2 \rvert$	GT pairs	ER task
FEBRL-1	Synthetic patient records	Febrl generator [14]	1,000	1,000	500	Dirty ER
FEBRL-2	Synthetic patient records	Febrl generator [14]	5,000	5,000	1,934	Dirty ER
FEBRL-3	Synthetic patient records	Febrl generator [14]	5,000	5,000	6,538	Dirty ER
FEBRL-4	Synthetic patient records	Febrl generator [14]	5,000	5,000	5,000	Clean-Clean ER
Synthea	Synthetic EHR patients	Synthea generator [19]	5,660	6,228	500	Clean-Clean ER
MedMentions	PubMed entity mentions	MedMentions corpus [15]	22,248	22,248	22,248	Clean-Clean ER
CMS	Medicare physician records	CMS Open Payments [16]	64,177	64,177	64,177	Clean-Clean ER
UMLS	Medical ontology concepts	UMLS MRCONSO.RRF [18]	134,992	156,932	643,166	Clean-Clean ER
RxNorm	Drug terminology (IN vs. BN)	NLM RxNorm CPC [20]	808	868	868	Clean-Clean ER

All collections are public: Febrl, Synthea and CMS need no registration, MedMentions is CC0, and UMLS and RxNorm require a free NLM Metathesaurus License. Derived collections, ground truth and preparation scripts are in our repository.

FEBRL-1 to FEBRL-3 are Dirty ER, with D_1 and D_2 the same file under increasing synthetic corruption: ground-truth pairs grow faster than the record count, so FEBRL-3 has 6,538 pairs among 5,000 records, indicating duplicate clusters larger than pairs. FEBRL-4 instead pairs two independently generated, clean 5,000-record populations with exactly one true match per record, making it Clean-Clean despite the shared family. Synthea likewise uses two populations: D_1 (5,660 records) unmodified, and D_2 (6,228 records) into which 500 noisy copies of D_1 records were injected as the ground truth. Despite this Dirty-ER-style noise injection, matches occur only across D_1 and D_2, never within either side, so Synthea is structurally Clean-Clean. The remaining four are Clean-Clean by construction: MedMentions links PubMed mention spans to their canonical concepts, CMS links two physician-record extracts, and UMLS and RxNorm link two source vocabularies, in each case with matches defined only across collections.

These datasets stress-test filtering along three axes underrepresented in general-purpose ER benchmarks:

- **Structured vs. unstructured records.** FEBRL and Synthea have fielded attributes (name, date of birth, address), whereas MedMentions consists of free-text mention spans, requiring schema-agnostic filtering.
- **Short, synonym-dense strings.** UMLS and RxNorm entities are often single or few-word concept names (drug ingredient vs. brand name), where token overlap between true matches can be low and between non-matches high.

– **Scale and duplicate density.** Inputs range from 808 records (RxNorm) to over 150K (UMLS). UMLS's 643K ground-truth pairs exceed either collection, reflecting a many-to-many mapping, and are two orders of magnitude beyond any other dataset, a major driver of its difficulty (Section 5).

All datasets are processed under *schema-agnostic* settings: filtering operates on the concatenation of all attribute values rather than a single distinguished attribute. This follows Neuhof et al. [2] for their datasets with low schema-aware attribute coverage, and reflects MedMentions and UMLS, where no single attribute reliably identifies an entity across both sides of the match.

4 Methodology

4.1 Problem Definition and Evaluation Metrics

Given two entity collections D_1 and D_2 (or a single collection D for Dirty ER, treated as $D_1 = D_2 = D$), a filtering technique produces candidate pairs $C \subseteq D_1 \times D_2$ containing as many true duplicates (the ground truth GT) as possible, while keeping $|C|$ small enough for subsequent pairwise matching to remain tractable. We report the standard filtering-effectiveness measures:

$$PC = \frac{|C \cap GT|}{|GT|}, \qquad PQ = \frac{|C \cap GT|}{|C|}, \qquad F_1 = \frac{2 \cdot PC \cdot PQ}{PC + PQ} \tag{1}$$

where PC (Pair Completeness) is recall over the ground truth, PQ (Pair Quality) is precision over the candidate set, and F_1 their harmonic mean. We additionally report wall-clock run time.

4.2 Blocking Workflows

We evaluate two configurations of the JedAI blocking pipeline, both following the four-stage workflow of block building, purging, filtering, and meta-blocking (comparison cleaning):

– **Parameter-Free.** Standard Blocking for FEBRL, Synthea, MedMentions and CMS; Q-Grams Blocking ($q = 6$) for UMLS and RxNorm, whose short concept strings yield degenerate token blocks. Comparison-based Block Purging removes oversized blocks (smoothing factor 1.0) and Comparison Propagation removes redundant comparisons, requiring no dataset-specific tuning.
– **Fine-Tuned.** We grid-search block builders (Standard; Q-Grams, $q \in \{2, \ldots, 6\}$), purging (on/off), block filtering ratios, and four meta-blocking algorithms (WEP, CEP, (Reciprocal) CNP, BLAST) with multiple edge-weighting schemes (ARCS, CBS, JS, EJS, PEARSON_X2), matching the configuration space of the reproduction study in Section 5. We report the configuration maximizing F_1 directly, not the fixed-recall protocol ($PC \geq \tau$, $\tau \in \{0.85, 0.90, 0.95\}$) of the base benchmark; for CMS and UMLS, whose scale makes the full grid impractical, we search a reduced grid of 3 meta-blocking methods, 3 weighting schemes and 3 filtering ratios.

4.3 Sparse Nearest-Neighbor Methods

ε-Join. We implement a schema-agnostic inverted-index similarity join: entities are tokenized, an index is built over the source collection, and for each target entity, candidates sharing at least one token are scored by cosine or Jaccard similarity and retained above a per-dataset threshold ε. Character trigrams are used for the short, synonym-dense UMLS, RxNorm and MedMentions vocabularies, whitespace tokens elsewhere; Table 5 lists the threshold, similarity and tokenizer per dataset.

kNN-Join. We additionally implement the top-k variant, retrieving for each query entity the k candidates with highest token-based similarity (ties at the kth score retained). We evaluate $k \in \{1, 5, 10, 20, 50, 100\}$ and select the highest-F_1 configuration per dataset. Tokenizers follow the same pattern as ε-Join; CMS uses Jaccard similarity, the rest cosine. Table 6 reports the selected k and effectiveness scores.

4.4 Dense Nearest-Neighbor Methods

All dense NN methods operate on 384-dimensional sentence embeddings from the pre-trained `all-MiniLM-L6-v2` model [13], applied to the concatenated attribute values of each entity, with no domain-specific fine-tuning.

- **FAISS.** An exact `IndexFlatIP` (inner-product search over L_2-normalized vectors, equivalent to exact cosine search) and an approximate `IndexHNSWFlat` ($M = 32$, $ef\_construction = 200$), for $k \in \{1, 5, 10, 20, 50, 100\}$.
- **ScaNN.** An asymmetric-hashing (AH) index using dot-product scoring on normalized vectors, with leaves and training sample size scaled to the collection (Section 5.4).
- **DeepBlocker.** The AutoEncoder (unsupervised reconstruction), CTT (self-supervised Cross-Tuple Training) and Hybrid (concatenation of both) tuple-embedding variants, each with exact top-k pairing, for $k \in \{1, 5, 10, 50\}$.

4.5 Experimental Environment

Blocking and ε-/kNN-Join experiments run in Java on JedAI (`-Xmx12g` for CMS and UMLS); dense NN experiments run in Python on a SLURM-managed GPU/CPU cluster. Since the two differ in language and hardware, we follow the ANN-Benchmarks methodology [17] of comparing *implementations* rather than *algorithms*, restricting cross-family run-time comparisons to relative orders of magnitude.

5 Results

5.1 Reproduction on General-Purpose Datasets

Table 2 and Table 3 report our reproduction of the blocking-workflow and kNN-Join baselines from the Continuous Filtering Benchmark [1,2] on its original ten datasets, using their published per-dataset configurations. The kNN-Join setup is notably heterogeneous: k ranges from 1 on six datasets to 26 on Amazon-Google Products, and character n-gram size varies from 2 to 5, evidence of the sparse family's sensitivity to tuning. Both tables match the base benchmark's trends: high effectiveness on clean bibliographic data (F_1 above 0.9 on DBLP-ACM and DBLP-Scholar) and sharp precision drops on noisy or many-to-one vocabularies, where Amazon-Google Products needs $k = 26$ to reach 0.900 recall and lands at 0.028 precision. This validates that our JedAI-based pipeline is correctly configured before we apply it to medical data.

Table 2. Blocking workflow reproduction on the ten general-purpose ER datasets. All use Standard Blocking; BF is the block filtering ratio; purging is comparison-based, smoothing factor 1.0. CNP = Cardinality Node Pruning, RCNP = Reciprocal CNP.

Dataset	Purging	BF	Meta-blocking (weighting)	PC	PQ	F_1
Restaurant	no	0.050	WEP (ARCS)	1.000	0.533	0.695
Abt-Buy	no	0.875	BLAST (PEARSON_X2)	0.902	0.216	0.349
Amazon-GP	yes	0.925	CNP (PEARSON_X2)	0.901	0.017	0.033
DBLP-ACM	yes	0.225	RCNP (EJS)	0.903	0.957	0.929
IMDb-TMDb	yes	1.000	RCNP (CBS)	0.922	0.382	0.540
IMDb-TVDb	no	0.975	RCNP (CBS)	0.924	0.189	0.314
TMDb-TVDb	no	0.350	RCNP (CBS)	0.909	0.154	0.264
Walmart-Amazon	no	0.225	RCNP (ARCS)	0.904	0.117	0.207
DBLP-Scholar	yes	0.625	RCNP (JS)	0.915	0.470	0.621
IMDb-DBpedia	no	0.800	BLAST (PEARSON_X2)	0.904	0.475	0.623

Table 3. kNN-Join reproduction on the ten general-purpose ER datasets, using the base benchmark's per-dataset configuration. k is per target-collection query entity.

Dataset	Tokenizer	Similarity	k	PC	PQ	F_1
Restaurant	Char 4-grams (multiset)	Dice	1	1.000	0.224	0.366
Abt-Buy	Char 3-grams (multiset)	Cosine	4	0.924	0.229	0.367
Amazon-GP	Char 5-grams (multiset)	Cosine	26	0.900	0.028	0.055
DBLP-ACM	Char 2-grams (multiset)	Cosine	1	0.996	0.954	0.974
IMDb-TMDb	Char 5-grams	Cosine	1	0.961	0.305	0.463
IMDb-TVDb	Char 5-grams	Cosine	1	0.910	0.122	0.215
TMDb-TVDb	Char 5-grams	Cosine	1	0.972	0.130	0.229
Walmart-Amazon	Char 4-grams (multiset)	Cosine	2	0.910	0.150	0.258
DBLP-Scholar	Char 4-grams	Cosine	1	0.957	0.877	0.915
IMDb-DBpedia	Char 4-grams	Cosine	5	0.903	0.149	0.256

5.2 Blocking Workflows on Medical Data

Table 4 reports the parameter-free and fine-tuned blocking workflows on all nine medical datasets. The parameter-free workflow alone exposes a clear pattern: it is competitive when attribute values are long and low-noise (FEBRL-4 at 0.907, MedMentions at 0.949), and collapses on the more heavily corrupted FEBRL-2 and FEBRL-3, and on UMLS, where Q-Gram blocks are simultaneously too small for adequate recall and too numerous for adequate precision. RxNorm underperforms differently: recall is high (PC=0.931) but precision only 0.075, because a single active ingredient generates many surface forms sharing Q-Grams with unrelated products.

Table 4. Blocking workflows on medical datasets: parameter-free (PF) vs. fine-tuned (FT), with the best grid-search configuration. BF = block filtering ratio, WEP = Weighted Edge Pruning, RCNP = Reciprocal Cardinality Node Pruning.

		Parameter-free			**Fine-tuned**		
Dataset	**Best FT configuration**	**PC**	**PQ**	**F_1**	**PC**	**PQ**	**F_1**
FEBRL-1	Standard, BF=0.30, WEP	1.000	0.240	0.387	0.998	0.250	0.400
FEBRL-2	Q-Grams (q=4), BF=0.35, BLAST	0.116	0.026	0.043	0.511	0.112	0.184
FEBRL-3	Q-Grams (q=4), BF=0.35, BLAST	0.103	0.102	0.102	0.451	0.166	0.242
FEBRL-4	Standard, BF=0.20, BLAST	0.993	0.835	0.907	1.000	1.000	1.000
Synthea	Standard, purging, BF=0.10, WEP	1.000	0.007	0.014	0.986	0.978	0.982
MedMentions	Standard, purging, BF=0.35, RCNP	1.000	0.903	0.949	1.000	0.994	0.997
CMS	Standard, BF=0.20, RCNP*	0.901	0.077	0.141	0.966	0.888	0.925
UMLS	Q-Grams (q=6), purging, BF=0.10, RCNP*	0.155	0.001	0.002	0.134	0.338	0.191
RxNorm	Q-Grams (q=5), BF=0.35, BLAST	0.931	0.075	0.139	0.909	0.822	0.863

* Reduced grid (3 meta-blocking methods, 3 weighting schemes, 3 BF settings); the full grid was terminated.

Fine-tuning improves every dataset, transformatively on three: Synthea by 70×, CMS by 6.6× and RxNorm by 6.2×. Once tuned, even tasks with very small ground truth relative to search space (Synthea: 500 pairs against 11,888 profiles) can be filtered almost perfectly. Two datasets resist tuning: FEBRL-1 moves only from 0.387 to 0.400, UMLS reaches only 0.191. In both, block-building is the bottleneck: no grid setting makes corrupted or synonymous profiles reliably co-occur, leaving downstream pruning ineffective. This is the first departure from the base benchmark, where tuned blocking workflows are consistently strongest.

The PC columns qualify this. On UMLS, RxNorm and Synthea, fine-tuning raises F_1 partly by trading recall away (UMLS PC falls from 0.155 to 0.134). Because filtering recall bounds the recall of the whole ER pipeline, an F_1-maximizing grid search can select configurations that a deployed system would reject, which is precisely what the base benchmark's fixed-recall protocol ($PC \geq \tau$) is designed to avoid.

5.3 Sparse Nearest-Neighbor Methods: ε-Join and kNN-Join

Unlike blocking, ε-Join requires no meta-blocking stage, trading a smaller configuration space for less flexibility in the precision-recall trade-off. Its behavior is strongly bimodal. On the FEBRL family it runs high-recall and low-precision (PC above 0.94, PQ between 0.21 and 0.36), whereas on MedMentions, CMS and UMLS it inverts, reaching precision of 0.99 or higher while retrieving under a fifth of true matches on the latter two. CMS is clearest: at $\varepsilon = 0.70$ all 6,287 retained pairs are correct, but 90% of the ground truth never enters the candidate set. A single global threshold cannot serve both regimes, the method's central limitation here.

Table 5. ε-Join on medical datasets, with per-dataset configuration.

Dataset	Tokenizer	Similarity	ε	PC	PQ	F_1
FEBRL-1	Whitespace	Cosine	0.50	0.996	0.250	0.399
FEBRL-2	Whitespace	Cosine	0.50	0.958	0.213	0.348
FEBRL-3	Whitespace	Cosine	0.50	0.946	0.356	0.517
FEBRL-4	Whitespace	Cosine	0.50	0.994	1.000	0.997
Synthea	Whitespace	Cosine	0.40	1.000	0.026	0.051
MedMentions	Character trigrams	Cosine	0.60	0.573	0.986	0.725
CMS	Whitespace	Jaccard	0.70	0.098	1.000	0.178
UMLS	Character trigrams	Cosine	0.60	0.185	0.925	0.308
RxNorm	Character trigrams	Cosine	0.50	0.624	0.221	0.327

Table 6. kNN-Join on medical datasets. The selected value of k maximizes F_1 over $k \in \{1, 5, 10, 20, 50, 100\}$. UMLS results use the ground-truth-complete 50% subset.

Dataset	Tokenizer	Similarity	k	PC	PQ	F_1
FEBRL-1	Whitespace	Cosine	1	1.000	1.000	1.000
FEBRL-2	Whitespace	Cosine	1	0.590	0.169	0.263
FEBRL-3	Whitespace	Cosine	1	0.533	0.671	0.594
FEBRL-4	Whitespace	Cosine	1	1.000	1.000	1.000
Synthea	Whitespace	Cosine	1	1.000	0.079	0.146
MedMentions	Character trigrams	Cosine	1	0.940	0.938	0.939
CMS	Whitespace	Jaccard	1	0.962	0.665	0.787
UMLS (50% subset)	Character trigrams	Cosine	5	0.552	0.512	0.531
RxNorm	Character trigrams	Cosine	1	0.812	0.864	0.837

kNN-Join varies the neighbors retained per query entity, and Table 6 shows that $k = 1$ maximizes F_1 on eight of nine datasets, as increasing k preserves

or improves PC while reducing PQ roughly proportionally. The exception is the 50% UMLS subset, where raising k from 1 to 5 lifts PC from 0.202 to 0.552 at a small enough precision cost to yield the best observed F_1 of 0.531. This is the signature of a dataset where the correct match often ranks second or third, as expected when one concept has many near-equivalent surface forms competing for the top position. kNN-Join is perfect on FEBRL-1 and FEBRL-4 and strong on MedMentions, RxNorm and CMS, but weaker on the noisier FEBRL-2 and FEBRL-3.

5.4 Dense Nearest-Neighbor Methods: FAISS, ScaNN, and DeepBlocker

Table 7 reports the best F_1 achieved by FAISS (Flat and HNSW) and ScaNN (AH) on each medical dataset. Unlike the general-purpose setting, approximation is not uniformly cheap here. HNSW stays within two points of Flat on seven of the nine datasets, but on CMS it collapses from 0.859 to 0.309, and ScaNN's asymmetric hashing collapses the same way to 0.288. This is not a retrieval-budget effect: HNSW recall on CMS is 0.309 at $k = 1$ and still only 0.375 at $k = 100$, so a larger candidate set does not recover the missing matches. Two independently implemented indices saturating at nearly the same recall points to the embedding geometry rather than either index: CMS profiles are short physician-name strings whose sentence embeddings cluster too tightly for the HNSW proximity graph or ScaNN's anisotropic partitioning to separate locally. The choice of index is therefore not orthogonal to effectiveness on medical data, and the penalty falls on the largest collection, where approximate retrieval is also the only affordable option (Table **??**).

Table 7. Dense NN best F_1 per medical dataset over $k \in \{1, 5, 10, 20, 50, 100\}$, using 384-dimensional `all-MiniLM-L6-v2` embeddings. F_1 decreases monotonically in k beyond k^*. Only F_1 is shown: at $k^* = 1$ on the datasets where $|D_1| = |D_2| = |GT|$ each query returns exactly one candidate, so PC = PQ = F_1 by construction.

Dataset	k^*	FAISS Flat	FAISS HNSW	ScaNN AH
FEBRL-1	1	0.955	0.955	0.819
FEBRL-2	1	0.315	0.315	0.285
FEBRL-3	1	0.551	0.550	0.497
FEBRL-4	1	0.931	0.917	0.730
Synthea	1	0.162	0.160	0.159
MedMentions	1	0.656	0.637	0.569
CMS	1	0.859	0.309	0.288
UMLS*	5	0.415	0.410	0.399
RxNorm	1	0.597	0.594	0.547

* UMLS values come from a separate run on the full collections; the other eight rows use the run described in Section 4.

DeepBlocker's medical evaluation is incomplete and its precision measurements unusable, so we report it separately in Table 8 as a qualitative, recall-focused data point rather than a comparable F_1 score:

- The `candidates` field is logged as 0.0 in every run, forcing precision (PQ) to 0.0 even where recall is high. We therefore report only the best recall (PC) per dataset, marked ‡ throughout.
- The **Hybrid** variant fails on *every* dataset with an identical matrix-shape error (`mat1 and mat2 shapes cannot be multiplied (256x150, 300x300)`), indicating a fixed embedding-dimension mismatch rather than a data-specific issue. It is omitted from Table 8.
- The run terminated during MedMentions, after AutoEncoder had completed all k but before CTT began, leaving the result file marked `.partial` and CMS and UMLS with no results. CTT is roughly eight times slower than AutoEncoder on the smaller collections, which points to a resource limit on the 22,248-profile MedMentions run.

Table 8. DeepBlocker best recall (PC‡) per dataset and variant; precision is unavailable (see text). PC is non-decreasing in k, so the reported k is the smallest attaining the maximum over $k \in \{1, 5, 10, 50\}$, unlike the F_1-maximizing k elsewhere. "–" marks no result.

Dataset	AE k	AE PC	CTT k	CTT PC
FEBRL-1	5	1.000	10	1.000
FEBRL-2	50	0.997	50	0.999
FEBRL-3	50	0.996	50	0.997
FEBRL-4	50	0.998	50	0.999
Synthea	1	1.000	1	1.000
MedMentions	50	0.556	–	–
CMS	–	–	–	–
UMLS	–	–	–	–
RxNorm	50	0.910	50	0.911

6 Discussion

Table 9 consolidates the best result per dataset and method family.

No single filtering paradigm dominates. As in the base benchmark [1,2], the best family is dataset-dependent even within one domain. Fine-tuned blocking, ε-Join and kNN-Join each take at least one dataset outright: blocking leads on Synthea, MedMentions, CMS and RxNorm and ties FEBRL-4; kNN-Join at $k = 1$ leads on FEBRL-1, FEBRL-3 and UMLS; ε-Join on FEBRL-2. A medical ER pipeline therefore cannot select a filtering family a priori from general-purpose results, which argues for either an ensemble of filters or a lightweight per-dataset selection step.

Table 9. Best F_1 per dataset and method family; per-metric values are in Tables 4 to 7. FAISS entries are the better of Flat and HNSW (Flat except on Synthea); dense and kNN-Join entries are at k^*; DeepBlocker entries are best PC (marked ‡), since PQ is unavailable (Table 8). Bold marks the best comparable F_1 per row.

Dataset	Block. (PF)	Block. (FT)	ε-Join	kNN-Join	FAISS	ScaNN	DeepBlocker (PC)
FEBRL-1	0.387	0.400	0.399	**1.000**	0.955	0.819	1.000‡
FEBRL-2	0.043	0.184	**0.348**	0.263	0.315	0.285	0.999‡
FEBRL-3	0.102	0.242	0.517	**0.594**	0.551	0.497	0.997‡
FEBRL-4	0.907	**1.000**	0.997	**1.000**	0.931	0.730	0.999‡
Synthea	0.014	**0.982**	0.051	0.146	0.162	0.159	1.000‡
MedMentions	0.949	**0.997**	0.725	0.939	0.656	0.569	0.556‡
CMS	0.141	**0.925**	0.178	0.787	0.859	0.288	not reached
UMLS	0.002	0.191	0.308	**0.531**	0.415	0.399	not reached
RxNorm	0.139	**0.863**	0.327	0.837	0.597	0.547	0.911‡

(i) All UMLS entries use the full collections except kNN-Join, run on a ground-truth-complete 50% subset ($|D_1| = 67{,}188$, $|D_2| = 78{,}466$, 323,550 pairs). (ii) Synthea was regenerated between the blocking and ε-Join runs ($|D_2| = 6{,}228$) and the nearest-neighbor runs ($|D_2| = 8{,}026$); the 500 ground-truth pairs are unchanged, but the roughly 30% smaller search space slightly favors the former in PQ.

Fine-tuning is necessary but not sufficient. It improves blocking on all nine datasets, and the 70× gain on Synthea and 6.6× on CMS dwarf the differences between families: for a fixed engineering budget, configuration search on one well-understood method is often worth more than adding untuned families. Yet the tuned workflow matches or exceeds every other family on only five datasets. On the three dirty FEBRL variants and on UMLS it is beaten, on FEBRL-1 decisively, by kNN-Join. This qualifies rather than contradicts the base benchmark's finding that tuned blocking is most robust: blocking loses where attribute values are short and character-corrupted, so no grid point yields blocks both small enough for precision and large enough for recall.

Sparse representations outperform dense ones on seven of nine datasets. Dense retrieval wins outright only on CMS, whose near-1-to-1 physician-name structure suits fixed-k retrieval and whose name variants embeddings capture better than Jaccard tokenization, and marginally on Synthea, where both families are weak. This mirrors the explanation Neuhof et al. [2] give for general-purpose data: all-MiniLM-L6-v2 is trained on web and news text, so medical terminology is underrepresented and embedding similarity is a noisier proxy for duplication than character overlap. The sharpest evidence is internal to the sparse family. On RxNorm, ε-Join and kNN-Join use identical character-trigram representations yet score 0.327 and 0.837, so the gap is attributable to the retrieval rule, not the representation: a single global threshold cannot serve a vocabulary in which ingredient and brand names vary widely in length, whereas fixed-k retrieval adapts per entity. Confirming the domain-vocabulary hypothesis would require a domain-specific embedding baseline (BioBERT, SapBERT), which we did not run.

UMLS is hard for every method. The best result is kNN-Join's $F_1 = 0.531$ on the 50% subset; on the full collections nothing exceeds 0.415. UMLS compounds three difficulties from Section 3: scale (134,992 × 156,932), many-to-many concept mapping (643,166 pairs), and short synonym-dense strings. It is also the only dataset where run time becomes a bottleneck in its own right (Table **??**). We regard UMLS-scale ontology matching as future work rather than a case general-purpose filtering can be expected to solve.

DeepBlocker needs engineering hardening before fair comparison. The uniform `candidates=0` defect, the deterministic Hybrid failure, and the run terminating during MedMentions make our numbers a lower bound rather than an assessment. This matches the base benchmark's observation that learned tuple embeddings are harder to configure than sparse counterparts, because stochastic components make failures less visible than a miscalibrated threshold.

Limitations. We evaluate only schema-agnostic settings, though the base benchmark shows schema-aware settings can change which family wins, and all dense methods use a single general-purpose embedding model. Synthea's duplicates are generated by deleting one character from a single name attribute, leaving all other fields identical, so every method attains near-perfect recall there and the low F_1 values of the untuned methods reflect precision alone; it should not be read as a hard corruption test. Finally, the two comparability caveats under Table 9 mean the UMLS and Synthea rows are not strictly like-for-like, and DeepBlocker remains incomplete on CMS and UMLS.

7 Conclusion

We extended the Continuous Filtering Benchmark to the medical domain, evaluating blocking workflows (parameter-free and fine-tuned), sparse nearest-neighbor methods (ε-Join, kNN-Join) and dense ones (FAISS, ScaNN, DeepBlocker) across nine datasets spanning synthetic patient records, electronic health records, biomedical text mentions and medical ontologies, after first validating our JedAI-based pipeline on the benchmark's original ten general-purpose datasets.

No single filtering family dominates. Fine-tuning yields larger gains than switching families, 70× in F_1 on Synthea and 6.6× on CMS, and syntactic token-based methods generally outperform general-purpose dense embeddings on medical vocabulary. Within the sparse family, the retrieval rule matters as much as the representation: on RxNorm, ε-Join and kNN-Join share a tokenizer yet score 0.327 and 0.837. On the full collections UMLS is the hardest case, combining scale, duplicate density and short synonym-dense strings. We also surfaced implementation issues invisible from aggregate numbers: a candidate-count logging defect and a deterministic Hybrid-model failure in DeepBlocker, and a collapse of both approximate indices on CMS that exact search does not share.

Future work. We plan to (1) re-run the comparison at fixed recall thresholds $\tau \in \{0.85, 0.90, 0.95\}$, following the base benchmark's protocol, since our

PC columns already show that F_1-maximizing configurations can trade recall away; (2) fix the DeepBlocker logging defect and Hybrid dimension mismatch and complete its evaluation on CMS and UMLS; (3) evaluate domain-specific biomedical embeddings (BioBERT, SapBERT) in place of all-MiniLM-L6-v2, to test whether the sparse-over-dense gap reflects vocabulary coverage rather than the dense paradigm itself; and (4) extend to schema-aware settings, where a single distinctive attribute (e.g., drug name in RxNorm) may change the ranking, as it does on general-purpose datasets.

References

1. Papadakis, G., Fisichella, M., Schoger, F., Mandilaras, G., Augsten, N., Nejdl, W.: Benchmarking filtering techniques for entity resolution. In: Proceedings of the 39th IEEE International Conference on Data Engineering (ICDE), pp. 653–666 (2023)
2. Neuhof, F., Fisichella, M., Papadakis, G., Nikoletos, K., Augsten, N., Nejdl, W., Koubarakis, M.: Open benchmark for filtering techniques in entity resolution. The VLDB Journal **33**(5), 1671–1696 (2024)
3. Papadakis, G., Skoutas, D., Thanos, E., Palpanas, T.: Blocking and filtering techniques for entity resolution: a survey. ACM Computing Surveys **53**(2), 31:1–31:42 (2021)
4. Papadakis, G., Koutrika, G., Palpanas, T., Nejdl, W.: Meta-blocking: taking entity resolution to the next level. IEEE Transactions on Knowledge and Data Engineering **26**(8), 1946–1960 (2014)
5. Simonini, G., Bergamaschi, S., Jagadish, H.V.: BLAST: a loosely schema-aware meta-blocking approach for entity resolution. Proceedings of the VLDB Endowment **9**(12), 1173–1184 (2016)
6. Papadakis, G., Mandilaras, G.M., Gagliardelli, L., Simonini, G., Thanos, E., Giannakopoulos, G., Bergamaschi, S., Palpanas, T., Koubarakis, M.: Three-dimensional entity resolution with JedAI. Information Systems **93**, 101565 (2020)
7. Randall, S.M., Ferrante, A.M., Boyd, J.H., Semmens, J.B.: Privacy-preserving record linkage on large real world datasets. Journal of Biomedical Informatics **50**, 205–212 (2014)
8. Liu, F., Shareghi, E., Meng, Z., Basaldella, M., Collier, N.: Self-alignment pretraining for biomedical entity representations. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), pp. 4228–4238 (2021)
9. Mann, W., Augsten, N., Bouros, P.: An empirical evaluation of set similarity join techniques. Proceedings of the VLDB Endowment **9**(9), 636–647 (2016)
10. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. IEEE Transactions on Big Data **7**(3), 535–547 (2021)
11. Guo, R., Sun, P., Lindgren, E., Geng, Q., Simcha, D., Chern, F., Kumar, S.: Accelerating large-scale inference with anisotropic vector quantization. In: Proceedings of the 37th International Conference on Machine Learning (ICML) (2020)
12. Thirumuruganathan, S., Li, H., Tang, N., Ouzzani, M., Govind, Y., Paulsen, D., Fung, G., Doan, A.: Deep learning for blocking in entity matching: a design space exploration. Proceedings of the VLDB Endowment **14**(11), 2459–2472 (2021)
13. Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992 (2019)

14. Christen, P.: Febrl – a freely available record linkage system with a graphical user interface. In: Proceedings of the 2nd Australasian Workshop on Health Data and Knowledge Management, pp. 17–25 (2008)
15. Mohan, S., Li, D.: MedMentions: a large biomedical corpus annotated with UMLS concepts. In: Proceedings of the Automated Knowledge Base Construction Conference (AKBC) (2019)
16. Centers for Medicare & Medicaid Services: Open Payments Data. https://openpaymentsdata.cms.gov/, accessed 2026-08-27
17. Aumüller, M., Bernhardsson, E., Faithfull, A.: ANN-Benchmarks: a benchmarking tool for approximate nearest neighbor algorithms. Information Systems **87**, 101374 (2020)
18. Bodenreider, O.: The Unified Medical Language System (UMLS): integrating biomedical terminology. Nucleic Acids Research **32**(suppl_1), D267–D270 (2004)
19. Walonoski, J., Kramer, M., Nichols, J., Quina, A., Moesel, C., Hall, D., Duffett, C., Dube, K., Gallagher, T., McLachlan, S.: Synthea: an approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. Journal of the American Medical Informatics Association **25**(3), 230–238 (2018)
20. Nelson, S.J., Zeng, K., Kilbourne, J., Powell, T., Moore, R.: Normalized names for clinical drugs: RxNorm at 6 years. Journal of the American Medical Informatics Association **18**(4), 441–448 (2011)